

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño

Definir la arquitectura de despliegue que permita una ejecución distribuida, segura y escalable del sistema DashKPI, garantizando la separación entre el frontend, backend, base de datos, servicios de procesamiento y observabilidad.

Orientación de la decisión

Adoptar una arquitectura basada en **nodos especializados interconectados** mediante protocolos seguros (HTTPS, SQL, Redis Pub/Sub) para distribuir las responsabilidades técnicas: interfaz, lógica de negocio, persistencia, y monitoreo.
Se estructura el sistema en capas físicas con contenedores o servidores dedicados para frontend, backend, base de datos, cache/mensajería y observabilidad.

Supuestos

El sistema DashKPI se desplegará en entornos con soporte para contenedores o máquinas virtuales (Docker / servidores dedicados).

Los módulos de KPIs, tareas, reportes y notificaciones ya están definidos a nivel lógico y de datos.

Se requiere mantener compatibilidad con Python 3.x, Django/DRF, React y MySQL.

Redis y Celery se emplean para tareas asíncronas y cacheo de datos.

Restricciones

El despliegue debe realizarse en infraestructura compatible con HTTPS y bases de datos SQL.

Los nodos deben comunicarse solo mediante interfaces REST o mensajería segura.

No se permitirá conexión directa entre cliente y base de datos.

La observabilidad debe estar integrada sin afectar el rendimiento general.

Requerimientos relacionados

RF-01: Consultar, crear y actualizar tareas y KPIs mediante API REST.

RF-02: Mostrar métricas e indicadores en dashboards React.

RF-03: Procesar y enviar notificaciones automáticas.

RNF-01: Escalabilidad horizontal del backend y workers.

RNF-02: Disponibilidad $\geq 95\%$.

RNF-03: Seguridad en la comunicación (HTTPS / autenticación por token).

Alternativas consideradas

Alternativa 1 Un único servidor que ejecuta frontend, backend y base de datos en la misma máquina.
Ventajas: Facilidad de despliegue y mantenimiento inicial.
Desventajas: Escalabilidad limitada, bajo aislamiento, riesgo de fallos generales.

Evaluación de alternativas

Alternativa 2 Separación básica entre frontend (cliente React) y backend (Django), pero sin servicios independientes para cache ni tareas asíncronas.
Ventajas: Mejor desempeño que el monolítico, fácil comunicación.
Desventajas: No permite procesamiento paralelo ni mensajería, poca tolerancia a fallos.

Alternativa n Despliegue multinodo según el UML:
 Frontend (React SPA), Backend (Django/DRF), Base de datos (MySQL), Workers (Celery), Cache/mensajería (Redis) y Observabilidad (Logs, métricas, alertas).
Ventajas: Modularidad, escalabilidad, extensibilidad y monitoreo integrado.
Desventajas: Mayor complejidad de configuración y consumo de recursos.

La Alternativa 1 (Monolítica) ofrece simplicidad inicial pero baja modularidad y escalabilidad, volviéndose poco viable a medida que crece el sistema.
 La Alternativa 2 (Cliente/Servidor) mejora el desempeño separando frontend y backend, aunque sigue limitada para procesamientos asíncronos o alto tráfico.
 Finalmente, la Alternativa 3 (Distribuida) proporciona mayor modularidad, rendimiento y capacidad de crecimiento, siendo la más adecuada para garantizar estabilidad, mantenimiento y expansión futura del sistema.

Alternativa seleccionada y Justificación

Se selecciona la Alternativa 3: Arquitectura distribuida.
 Proporciona aislamiento, escalabilidad horizontal y capacidad para integrar servicios asíncronos (Celery + Redis).
 Además, se alinea con la arquitectura de despliegue descrita en el UML DeployV2 y con las buenas prácticas de sistemas modernos basados en Django/React.

Decisiones relacionadas

Definir la tecnología y contenedores para cada nodo (Docker Compose o Kubernetes).
 Establecer políticas de comunicación segura (HTTPS y autenticación JWT).
 Diseñar monitoreo unificado en el nodo de observabilidad.

Comentarios

Esta decisión consolida la base para la automatización de despliegues continuos (CI/CD) y la futura integración con servicios en la nube (AWS/Azure).
 Permite extender DashKPI con microservicios adicionales como módulos de predicción o análisis avanzado de KPIs.